Migración de infraestructura física a entornos virtualizados

Gabriel Fernando Castillo Mendieta
Esteban Nicolás Peña Coronado
Luis Javier López Galindo
Johan Sebastián Gil Salamanca
Brayan Alejandro Cifuentes Quiroga

Docente: Frey Alfonso Santamaría Buitrago
Ingeniero de Sistemas

Universidad Pedagógica y Tecnológica de Colombia
Ingeniería de Sistemas y Computación
Electiva IaaS y Virtualización
Tunja
2026

Tabla de contenido

- Migración de infraestructura física a entornos virtualizados
  - Tabla de contenido
- 1. Investigación
  - 1.1 Virtualización: fundamentos y evolución histórica
  - 1.2 Hipervisores: arquitectura y clasificación
    - 1.2.1 Hipervisor de Tipo 1 (bare-metal)
    - 1.2.2 Hipervisor de Tipo 2 (hosted)
    - 1.2.3 Comparación de arquitecturas
  - 1.3 Proxmox VE: componentes y arquitectura interna
    - 1.3.1 KVM (Kernel-based Virtual Machine)
    - 1.3.2 QEMU como emulador de hardware
    - 1.3.3 Paravirtualización con VirtIO
  - 1.4 Oracle VirtualBox: arquitectura y características
  - 1.5 Modos de red en entornos virtualizados
    - 1.5.1 NAT (Network Address Translation)
    - 1.5.2 Adaptador puente (Bridge)
    - 1.5.3 Red interna (Internal Network)
    - 1.5.4 Solo-anfitrión (Host-Only)
  - 1.6 Benchmarking de infraestructura virtualizada
    - 1.6.1 Concepto de benchmark
    - 1.6.2 Métricas de rendimiento relevantes
  - 1.7 Apache HTTP Server en entornos virtualizados
  - 1.8 PostgreSQL en entornos virtualizados
  - 1.9 Implicaciones de seguridad según el tipo de hipervisor

- 2. Análisis y síntesis
  - 2.1 Proxmox VE y VirtualBox como representantes de cada arquitectura
  - 2.2 La elección de Apache y PostgreSQL como servicios del entorno
  - 2.3 La definición del modo de red como decisión del equipo
  - 2.4 El diseño del benchmark como solución a una limitación del entorno
- 3. Respuesta a las preguntas iniciales
  - 3.1. Nivel Conceptual
    - 3.1.1. Diferencias fundamentales entre hipervisores de Tipo 1 y Tipo 2
    - 3.1.2. Impacto en seguridad y aislamiento
    - 3.1.3. Implicaciones en la gestión y administración
    - 3.1.4. Proxmox VE como plataforma Tipo 1: componentes principales
  - 3.2. Nivel Técnico
    - 3.2.1. Instalación y configuración inicial de Proxmox y VirtualBox
    - 3.2.2. Modos de red: NAT, Bridge, Host-Only e Internal
    - 3.2.3. Instalación y configuración de Apache
    - 3.2.4. Interconexión de VMs gestionadas por distintos hipervisores
    - 3.2.5. Instalación y configuración de PostgreSQL
    - 3.2.6. Direccionamiento IP y configuración de interfaces de red
    - 3.2.7. Pruebas de conectividad entre VMs
  - 3.3. Nivel Comparativo
    - 3.3.1. Metodología aplicada para los benchmarks
    - 3.3.2. Resultados del benchmark de Apache
    - 3.3.3. Resultados del benchmark de PostgreSQL
    - 3.3.4. Interpretación de los resultados
  - 3.4. Nivel Estratégico
    - 3.4.1. Criterios de selección del hipervisor según tipo de servidor
    - 3.4.2. Costos de mantenimiento y soporte
    - 3.4.3. Recomendación estratégica para ACME
  - 3.5. Nivel Reflexivo
    - 3.5.1. Retos técnicos y de compatibilidad encontrados
    - 3.5.2. Lecciones aprendidas
  - 3.6. Respuesta a la pregunta inicial
- 4. Conclusiones
- Referencias

---

1. Investigación

La presente sección constituye el componente investigativo del proyecto, desarrollado en correspondencia con el paso 5 de la metodología de Aprendizaje Basado en Proyectos (ABP). Su propósito es dar respuesta sistemática a los vacíos de conocimiento identificados y construir el marco teórico que fundamenta las decisiones técnicas documentadas en el informe final.

Esta investigación aborda los siguientes ejes temáticos: fundamentos de virtualización, arquitecturas de hipervisores Tipo 1 y Tipo 2, funcionamiento de Proxmox VE y Oracle VirtualBox, modelos de red en

entornos virtualizados, metodología de benchmarking comparativo, y criterios estratégicos para la selección de plataformas de virtualización en distintos contextos.

1.1 Virtualización: fundamentos y evolución histórica

La virtualización es una tecnología que permite crear representaciones abstractas de recursos de cómputo (procesadores, memoria, almacenamiento y red), de manera que múltiples sistemas operativos o aplicaciones puedan ejecutarse de forma independiente y simultánea sobre un único hardware físico (Portnoy, 2012). Esta capacidad de abstracción transforma los recursos físicos en conjuntos de recursos lógicos administrables, desacoplando el software del hardware que lo ejecuta.

El concepto no es reciente. Sus orígenes se remontan a la década de 1960, cuando IBM desarrolló el sistema CP/CMS para sus mainframes de la serie 360/370, con el objetivo de permitir que múltiples usuarios compartieran simultáneamente los costosos recursos de cómputo disponibles (Creasy, 1981). No obstante, fue durante la década de 1990 cuando la virtualización comenzó a democratizarse en la arquitectura x86. VMware, fundada en 1998, resolvió los desafíos técnicos que presentaba la arquitectura x86 (originalmente no diseñada para ser virtualizada) y sentó las bases de la virtualización moderna para servidores (Smith & Nair, 2005).

En la primera década del siglo XXI, la adopción de la virtualización se aceleró en los centros de datos debido a la convergencia de varios factores: el aumento en la capacidad de los procesadores multinúcleo, la incorporación de extensiones de virtualización por hardware en arquitecturas de Intel (VT-x) y AMD (AMD-V), y la madurez de proyectos de código abierto como KVM y Xen (Laureano & Maziero, 2008). Actualmente, la virtualización es una base fundamental para tecnologías como la computación en la nube, la infraestructura como servicio (IaaS) y los centros de datos modernos.

Los beneficios que impulsan su adopción en entornos empresariales son diversos. En primer lugar, la consolidación de servidores permite ejecutar múltiples cargas de trabajo en un mismo equipo físico, lo que mejora el aprovechamiento de los recursos y reduce costos de hardware, energía y espacio (Portnoy, 2012). En segundo lugar, el aislamiento entre máquinas virtuales asegura que fallos o problemas de seguridad en una de ellas no afecten a las demás. Finalmente, herramientas de gestión como las instantáneas (snapshots), la clonación, la migración en caliente y la alta disponibilidad brindan una flexibilidad operativa difícil de alcanzar con servidores físicos tradicionales.

1.2 Hipervisores: arquitectura y clasificación

El componente de software que hace posible la virtualización es el hipervisor, también conocido como Virtual Machine Monitor (VMM). Su función es actuar como intermediario entre el hardware físico y las máquinas virtuales, gestionando el acceso a los recursos del sistema de forma controlada y aislada (Popek & Goldberg, 1974).

La clasificación más utilizada en la literatura fue propuesta por Goldberg (1972), quien distingue dos tipos arquitectónicos fundamentales.

1.2.1 Hipervisor de Tipo 1 (bare-metal)

Un hipervisor de Tipo 1 se instala y ejecuta **directamente sobre el hardware físico**, sin la intermediación de un sistema operativo anfitrión. Actúa él mismo como un sistema operativo minimalista cuya función principal es gestionar las máquinas virtuales y arbitrar el acceso de estas al hardware subyacente (Tanenbaum & Bos, 2015).

Ejemplos representativos incluyen VMware ESXi, Microsoft Hyper-V (en su modalidad bare-metal), Citrix Hypervisor (antes XenServer) y **Proxmox VE**. La arquitectura se ilustra en el siguiente esquema:

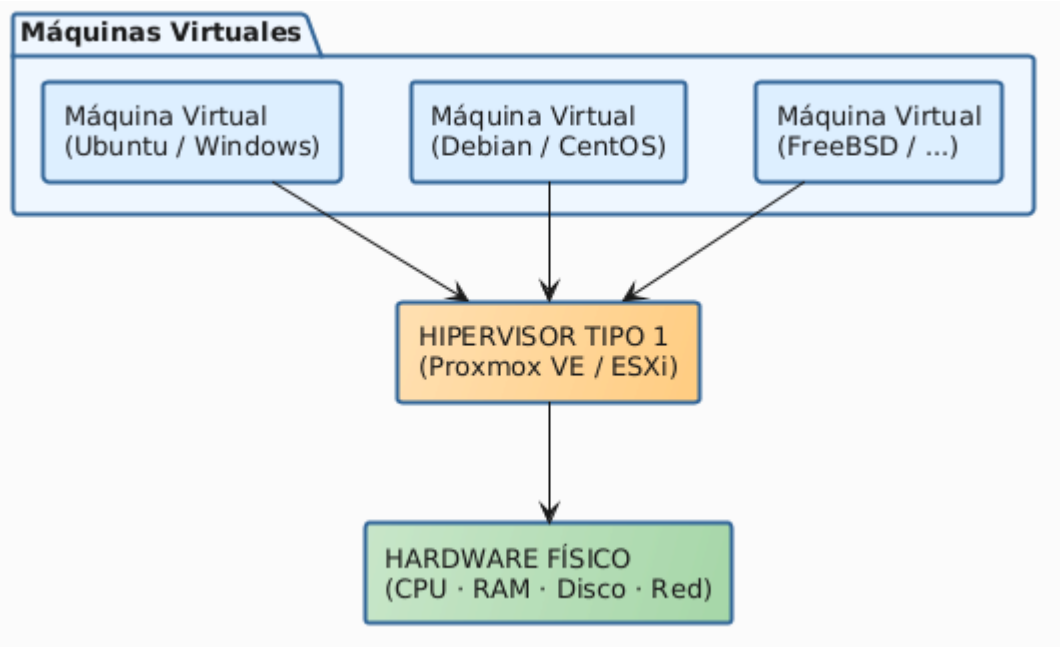

Figura 1. Arquitectura basica Hipervisor Tipo 1.

Las ventajas principales de este modelo son su menor latencia en el acceso al hardware (al eliminar la capa del SO anfitrión), su mayor eficiencia en la gestión de recursos y su superficie de ataque reducida desde el punto de vista de la seguridad (Laureano & Maziero, 2008).

1.2.2 Hipervisor de Tipo 2 (hosted)

Un hipervisor de Tipo 2 se instala como una **aplicación sobre un sistema operativo anfitrión** preexistente (Windows, Linux, macOS). Las máquinas virtuales se ejecutan como procesos dentro de dicho sistema operativo, que actúa como intermediario entre el hipervisor y el hardware (Tanenbaum & Bos, 2015).

Ejemplos representativos incluyen **Oracle VirtualBox**, VMware Workstation/Fusion y QEMU en su modalidad sin aceleración KVM.

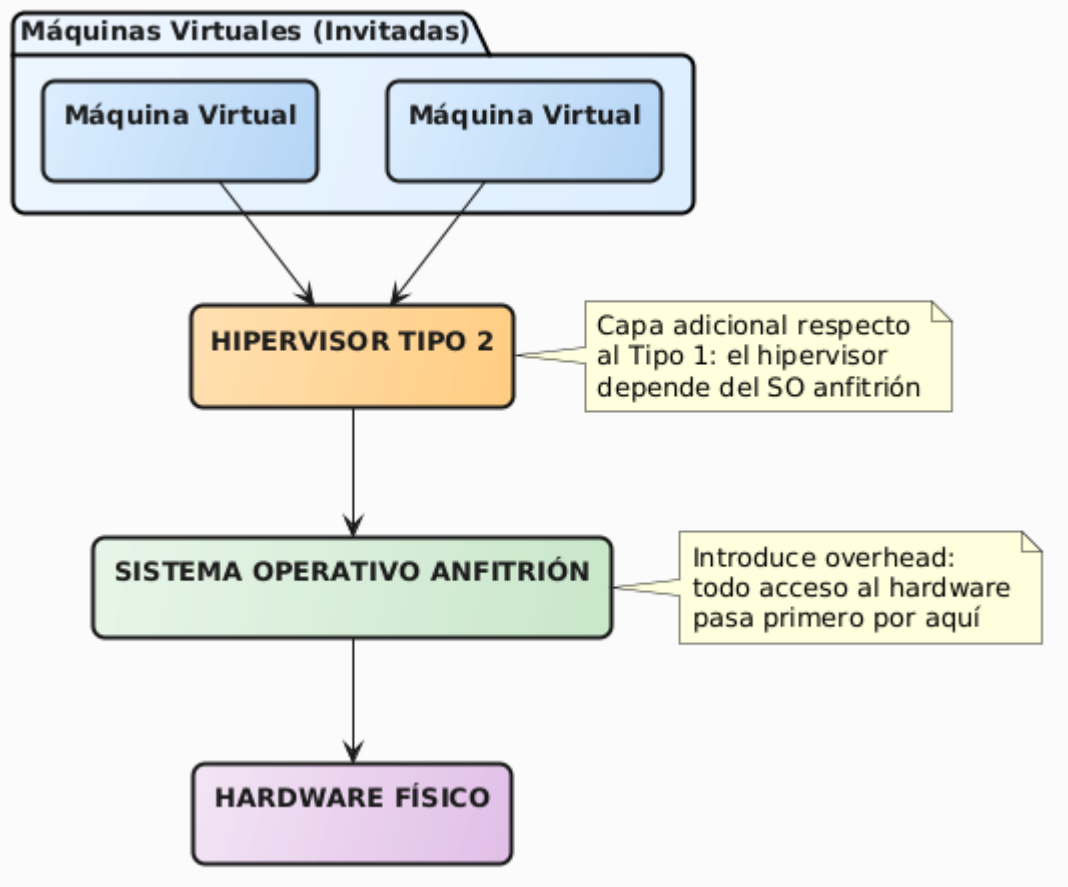

Figura 2. Arquitectura basica Hipervisor Tipo 2.

La principal desventaja es la capa adicional introducida por el sistema operativo anfitrión, que genera un *overhead* de rendimiento. No obstante, los hipervisores Tipo 2 son más fáciles de instalar, configurar y usar en estaciones de trabajo de desarrollo, lo que los hace idóneos para entornos de laboratorio, pruebas y desarrollo (Portnoy, 2012).

1.2.3 Comparación de arquitecturas

Tabla 1. Comparacion entre hipervisores tipo 1 y tipo 2.

Aspecto	Tipo 1 (Proxmox VE)	Tipo 2 (VirtualBox)
Instalación	Bare-metal, ocupa el disco completo	Sobre SO anfitrión existente
Overhead	Mínimo (acceso directo al HW)	Mayor (capa del SO anfitrión)
Rendimiento esperado	Superior en producción	Inferior, variable según carga del SO anfitrión
Facilidad de uso	Interfaz web; curva de aprendizaje mayor	GUI intuitiva; más accesible
Caso de uso típico	Producción, centros de datos, IaaS	Desarrollo, laboratorio, pruebas locales
Ejemplos	ESXi, Hyper-V, Proxmox, Xen	VirtualBox, VMware Workstation, Parallels

Aspecto	Tipo 1 (Proxmox VE)	Tipo 2 (VirtualBox)
Licencia	Frecuentemente comercial o open-source enterprise	Frecuentemente gratuito para uso personal

1.3 Proxmox VE: componentes y arquitectura interna

Proxmox Virtual Environment (Proxmox VE) es una plataforma de virtualización empresarial de código abierto basada en Debian GNU/Linux, desarrollada por Proxmox Server Solutions GmbH y publicada bajo licencia GNU AGPL v3 (Proxmox Server Solutions, 2024). Integra en un único paquete dos tecnologías de virtualización complementarias: KVM para máquinas virtuales completas y LXC (Linux Containers) para contenedores del sistema. Para el presente proyecto se utilizó Proxmox VE en su versión 9.1.

1.3.1 KVM (Kernel-based Virtual Machine)

KVM es un módulo del kernel de Linux que convierte al propio kernel en un hipervisor de Tipo 1. Fue integrado al kernel Linux mainline a partir de la versión 2.6.20 (enero de 2007) y desde entonces constituye la base de virtualización de la mayor parte de la infraestructura en la nube pública (incluyendo AWS, Google Cloud y OpenStack) (Kivity et al., 2007).

KVM aprovecha las extensiones de virtualización por hardware presentes en los procesadores modernos: **Intel VT-x** (Virtualization Technology) y **AMD-V** (AMD Virtualization). Estas extensiones introducen un nuevo modo de operación del procesador denominado *VMX root* (en Intel), desde el cual el hipervisor puede interceptar y gestionar las instrucciones privilegiadas emitidas por los sistemas operativos invitados sin incurrir en el costoso proceso de *trap-and-emulate* que caracterizaba a las implementaciones de software puro (Adams & Agesen, 2006).

Desde la perspectiva del sistema operativo invitado, KVM expone una CPU virtual (vCPU) que ejecuta instrucciones directamente en el hardware físico en modo no privilegiado. Las instrucciones que requieren acceso a recursos privilegiados son interceptadas por KVM, que las gestiona o las pasa al hardware de forma segura.

1.3.2 QEMU como emulador de hardware

KVM por sí solo gestiona la CPU y la memoria, pero no emula el hardware periférico (controladores de disco, tarjetas de red, etc.). Para esta función, Proxmox combina KVM con **QEMU** (Quick Emulator), un emulador de hardware completo de código abierto (Bartholomew, 2006).

La combinación KVM+QEMU resulta en un sistema donde la CPU virtual se ejecuta de forma casi nativa (gracias a KVM) mientras que el hardware periférico es emulado o paravirtualizado por QEMU. Esta arquitectura es la que permite a Proxmox ofrecer VMs con rendimiento cercano al de hardware físico para cargas de trabajo de CPU intensivas.

1.3.3 Paravirtualización con VirtIO

Un aspecto técnico relevante para comprender el rendimiento de la MV de Proxmox en este proyecto es el uso de **VirtIO**, un estándar de virtualización para dispositivos de E/S que permite a los sistemas operativos invitados comunicarse directamente con el hipervisor mediante una API optimizada, en lugar de emular hardware físico completo (Russell, 2008).

- **Disco VirtIO SCSI**: controlador de almacenamiento paravirtualizado, más eficiente que la emulación de un controlador SCSI físico convencional.
- **Red VirtIO (paravirtualized)**: interfaz de red paravirtualizada, con menor latencia que la emulación de una tarjeta Intel E1000.

La paravirtualización reduce el overhead de las operaciones de I/O al eliminar la capa de emulación de hardware, permitiendo que el SO invitado utilice *drivers* especializados que conocen que están siendo virtualizados y se comunican de forma más eficiente con el hipervisor (Russell, 2008).

1.4 Oracle VirtualBox: arquitectura y características

Oracle VirtualBox es un hipervisor de Tipo 2 de código abierto (GPLv2) mantenido por Oracle Corporation. Originalmente desarrollado por Innotek GmbH, fue adquirido por Sun Microsystems en 2008 y posteriormente por Oracle en 2010 (Oracle Corporation, 2024).

VirtualBox opera como una aplicación de espacio de usuario sobre el sistema operativo anfitrión. Para gestionar el acceso al hardware, instala un driver del kernel (`vboxdrv`) que actúa como puente entre la aplicación de VirtualBox y los recursos del hardware. Este driver es el que permite a VirtualBox aprovechar las extensiones Intel VT-x/AMD-V para ejecutar código de las VMs directamente en hardware, reduciendo el overhead en operaciones de CPU intensivas.

Una característica diferenciadora de VirtualBox respecto a otros hipervisores Tipo 2 es su portabilidad multiplataforma: puede ejecutarse sobre Windows, Linux, macOS y Solaris, lo que lo hace especialmente útil en entornos heterogéneos de desarrollo.

1.5 Modos de red en entornos virtualizados

La configuración de red es uno de los aspectos más críticos en el diseño de infraestructuras virtualizadas. Tanto VirtualBox como Proxmox ofrecen múltiples modos de red, cada uno con implicaciones distintas en cuanto a conectividad, aislamiento y rendimiento (Oracle Corporation, 2024; Proxmox Server Solutions, 2024).

1.5.1 NAT (Network Address Translation)

En el modo NAT, el hipervisor actúa como un router con traducción de direcciones. La VM obtiene una dirección IP privada en una subred gestionada exclusivamente por el hipervisor (típicamente `10.0.2.x` en VirtualBox). El tráfico saliente de la VM es mascarado por la IP del equipo anfitrión, permitiéndole acceder a Internet y a la red externa.

La limitación principal de NAT es que **las VMs no son accesibles desde el exterior** sin configurar redirección de puertos (*port forwarding*) explícita, y tampoco pueden comunicarse directamente entre sí cuando están en distintas instancias NAT. Por esta razón, NAT es adecuado para acceso a Internet desde una VM pero no para comunicación inter-VM.

1.5.2 Adaptador puente (Bridge)

En el modo Bridge, el hipervisor conecta la interfaz de red virtual de la VM directamente al switch virtual que interconecta el hardware físico de la red. Desde la perspectiva de la red, la VM aparece como un nodo

independiente con su propia dirección MAC y puede recibir una IP directamente del servidor DHCP de la red física.

Este modo opera a **nivel de capa 2 (Ethernet)** del modelo OSI. El hipervisor crea un bridge de software que une la interfaz física del equipo anfitrión con las interfaces virtuales de las VMs. El resultado es que la VM participa en la misma red que el equipo anfitrión, con acceso total a otros nodos de la red local.

El modo Bridge es el que se utilizó en este proyecto para lograr la comunicación entre las VMs de Proxmox y VirtualBox, y constituye la respuesta técnica al problema de conectividad entre hipervisores distintos (ver sección 5.6).

1.5.3 Red interna (Internal Network)

La red interna crea un segmento de red aislado que solo existe dentro del hipervisor. Las VMs conectadas a la misma red interna pueden comunicarse entre sí, pero no tienen acceso al equipo anfitrión ni a la red exterior. Es útil para simular redes privadas completamente aisladas, aunque requiere configuración manual de IP (no hay DHCP por defecto).

1.5.4 Solo-anfitrión (Host-Only)

El modo Host-Only crea una red virtual entre el equipo anfitrión y sus VMs. Las VMs pueden comunicarse entre sí y con el anfitrión, pero no tienen acceso a la red externa. VirtualBox crea una interfaz virtual en el anfitrión que actúa como gateway de esta red privada. Es frecuentemente utilizado para acceso SSH a VMs desde el equipo de desarrollo sin exponerlas a la red corporativa.

1.6 Benchmarking de infraestructura virtualizada

1.6.1 Concepto de benchmark

Un *benchmark* es un procedimiento de evaluación estandarizado que mide el rendimiento de un sistema bajo condiciones controladas y reproducibles, con el propósito de comparar dicho rendimiento contra un punto de referencia o contra sistemas alternativos (Gray, 1993). En el contexto de la infraestructura virtualizada, los benchmarks permiten cuantificar el impacto del hipervisor sobre el rendimiento de las cargas de trabajo, aislando las variables del entorno.

Un principio fundamental del benchmarking comparativo es la **equivalencia de condiciones**: para que la comparación sea válida, las VMs evaluadas deben tener configuraciones equivalentes de vCPU, RAM, sistema operativo y versiones de software.

1.6.2 Métricas de rendimiento relevantes

Las métricas seleccionadas para este proyecto cubren los tres subsistemas que más frecuentemente constituyen cuellos de botella en cargas de trabajo de servidor:

Rendimiento de CPU:

- *Tiempo de ejecución* (segundos): tiempo total para completar una operación intensiva de CPU.
- *Throughput* (operaciones/segundo): número de operaciones completadas por unidad de tiempo.

- Las pruebas de PostgreSQL (sort, agregaciones, join, funciones matemáticas) ejercen principalmente la CPU.

Rendimiento de I/O de disco:

- *IOPS (Input/Output Operations Per Second)*: número de operaciones de lectura/escritura por segundo.
- *Throughput de disco* (MB/s): volumen de datos transferidos por unidad de tiempo.
- *Latencia de I/O* (ms): tiempo desde que se solicita una operación hasta que se completa.
- Las pruebas de INSERT masivo y lectura de PostgreSQL ejercen principalmente el subsistema de almacenamiento.

Rendimiento de red y servidor web:

- *RPS (Requests Per Second)*: número de peticiones HTTP atendidas por segundo. Es la métrica principal de capacidad de un servidor web.
- *Latencia de respuesta* (ms): tiempo transcurrido desde que el cliente envía la petición hasta que recibe la respuesta completa.
- *Tasa de error*: porcentaje de peticiones que no reciben respuesta exitosa (HTTP 200).

1.7 Apache HTTP Server en entornos virtualizados

Apache HTTP Server es el servidor web de código abierto más ampliamente desplegado en el mundo desde 1996 (Netcraft, 2024). Es mantenido por la Apache Software Foundation y publicado bajo la licencia Apache 2.0. Su arquitectura modular permite extender sus capacidades mediante módulos cargables dinámicamente (DSO - Dynamic Shared Objects).

En entornos virtualizados, Apache se comporta de manera similar a como lo hace en hardware físico, con la diferencia de que el rendimiento de red y de I/O depende de la capa de virtualización. Los aspectos de Apache más relevantes para el benchmark son:

- **Modelo de procesamiento (MPM):** Apache utiliza por defecto en Ubuntu el módulo `event`, que gestiona las conexiones mediante un modelo de hilos y eventos asíncronos, permitiendo atender eficientemente múltiples conexiones concurrentes con un número limitado de hilos.
- **Archivos estáticos:** El servicio de archivos estáticos (como la página de bienvenida utilizada en el benchmark) depende principalmente de las capacidades de I/O del sistema de archivos subyacente y de la velocidad de la interfaz de red virtualizada.

1.8 PostgreSQL en entornos virtualizados

PostgreSQL es un sistema de gestión de bases de datos objeto-relacional (ORDBMS) de código abierto, reconocido por su robustez, extensibilidad y cumplimiento de los estándares ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) (PostgreSQL Global Development Group, 2024).

En entornos virtualizados, el rendimiento de PostgreSQL está fuertemente condicionado por dos subsistemas del hipervisor:

Subsistema de almacenamiento: PostgreSQL es intensivo en I/O de disco, especialmente en operaciones de escritura que involucran el mecanismo Write-Ahead Logging (WAL). La latencia de escritura en disco es el factor limitante principal para operaciones de INSERT masivo y UPDATE. En el benchmark del proyecto, la

diferencia de +280% en tiempos de INSERT entre la VM de Proxmox (HDD) y la de VirtualBox (SSD) ilustra dramáticamente este impacto.

Subsistema de CPU: Las operaciones de ordenamiento (ORDER BY), agregación estadística (AVG, STDDEV, percentiles) y joins analíticos consumen CPU de forma intensiva. La diferencia en rendimiento de CPU entre generaciones de procesadores (i5-3470 de 2013 vs. i5-12450H de 2022) introduce un factor de variación que va más allá de las diferencias entre hipervisores.

PostgreSQL expone dos archivos de configuración relevantes para el acceso remoto en este proyecto:

- `postgresql.conf`: controla los parámetros operativos del servidor, incluyendo `listen_addresses` que determina en qué interfaces de red acepta conexiones.
- `pg_hba.conf` (Host-Based Authentication): define las reglas de autenticación por dirección IP de origen, método de autenticación y base de datos. La entrada `host all all 192.168.137.0/24 scram-sha-256` permite conexiones autenticadas desde cualquier equipo del segmento de red del proyecto.

1.9 Implicaciones de seguridad según el tipo de hipervisor

El tipo de hipervisor tiene implicaciones directas sobre la seguridad de la infraestructura virtualizada, un aspecto que debe considerarse en la selección de plataforma para la empresa ACME.

Aislamiento entre VMs: Ambos tipos de hipervisor garantizan aislamiento entre las VMs mediante el mecanismo de virtualización del hardware (Intel VT-x/AMD-V). Sin embargo, la *superficie de ataque* difiere: un hipervisor Tipo 1 tiene menos código ejecutándose en el sistema (no hay SO anfitrión), lo que reduce los vectores de ataque potenciales (Laureano & Maziero, 2008). Un hipervisor Tipo 2 hereda todas las vulnerabilidades del SO anfitrión: si el SO es comprometido, todas las VMs están en riesgo.

VM Escape: El ataque más crítico en entornos virtualizados es el *VM Escape*, mediante el cual un atacante que comprometió una VM logra ejecutar código en el hipervisor o en el sistema anfitrión, obteniendo acceso a todas las demás VMs. Este tipo de ataque ha sido documentado en vulnerabilidades como CVE-2019-5544 (ESXi) y múltiples CVEs en VirtualBox. Los hipervisores Tipo 1 de grado empresarial (ESXi, Proxmox) tienen historiales más extensos de auditoría de seguridad y ciclos de parche más rápidos que los Tipo 2.

Gestión de acceso: Proxmox VE dispone de un sistema de control de acceso basado en roles (RBAC) que permite definir granularmente los permisos de usuarios sobre recursos específicos (nodos, VMs, almacenamiento). VirtualBox, orientado a uso individual, no dispone de un sistema de gestión de acceso equivalente, lo que lo hace menos adecuado en entornos multi-usuario de producción.

---

2. Análisis y síntesis

Esta sección documenta el proceso de análisis colectivo mediante el cual el equipo, apoyándose en el marco teórico construido en la investigación, examinó las razones detrás de cada herramienta y tomó las decisiones técnicas que orientarán la ejecución del proyecto. No se trata de analizar resultados obtenidos —eso corresponde a la sección 9— sino de explicar cómo la teoría informa las decisiones de diseño e implementación que se llevarán a cabo.

2.1 Proxmox VE y VirtualBox como representantes de cada arquitectura

El proyecto plantea desde el principio que la arquitectura debe incluir al menos un hipervisor de Tipo 1 y uno de Tipo 2. Esta exigencia no es arbitraria: la pregunta guía del nivel conceptual requiere comparar ambas arquitecturas con evidencia práctica, lo cual solo será posible si las dos están presentes y operando simultáneamente en la misma infraestructura. Sin esa dualidad, la comparación quedaría en el plano puramente teórico.

Para cubrir el Tipo 1, el equipo evaluó las opciones disponibles. VMware ESXi y Citrix Hypervisor son soluciones propietarias con licenciamiento comercial que no se justifica en el contexto del proyecto. Microsoft Hyper-V exige licencias de Windows Server. Proxmox VE, en cambio, es completamente open-source bajo licencia GNU AGPL v3, combina la solidez técnica de KVM+QEMU con una interfaz web de administración integrada, y es comparable en capacidades a soluciones empresariales de pago. Por estas razones, Proxmox VE 9.1 se instalará en modo bare-metal sobre el equipo HP con Intel Core i5-3470, dedicándolo por completo al hipervisor tal como lo exige la arquitectura Tipo 1.

Para cubrir el Tipo 2, el equipo eligió Oracle VirtualBox. Su portabilidad multiplataforma —funciona sobre Windows, Linux y macOS— y su disponibilidad gratuita para uso educativo lo hacen la opción más práctica para el equipo de desarrollo. A diferencia de VMware Workstation, que requiere licencia comercial, VirtualBox se instala como una aplicación estándar sobre el sistema operativo anfitrión sin requerir modificaciones al sistema ni arranque especializado. VirtualBox correrá sobre Pop!_OS 24.04 LTS en el equipo con Intel Core i5-12450H. Esta asignación responde también a una restricción de hardware: Proxmox necesita dedicar el equipo por completo, y el equipo HP era el único disponible para ese propósito.

En cuanto a los sistemas operativos de las máquinas virtuales, el equipo optó por Ubuntu Server 24.04.4 LTS para todas las VMs de servidor. Esta elección se fundamenta en que Ubuntu Server tiene soporte de largo plazo (LTS), repositorios actualizados con las versiones más recientes de Apache y PostgreSQL, y amplia documentación para configuraciones de red con Netplan, que será necesaria para asignar IPs estáticas en la VM de VirtualBox. Para la VM cliente se eligió Fedora Workstation, que al ser una distribución orientada a escritorio permite usar herramientas gráficas como DBeaver para conectarse a las bases de datos y un navegador web para verificar el funcionamiento de Apache, lo cual facilita la validación del entorno desde una perspectiva de usuario final.

2.2 La elección de Apache y PostgreSQL como servicios del entorno

El proyecto establece que la infraestructura de ACME requiere un servidor web y un servidor de base de datos. A partir de esa definición, el equipo analizó qué software concreto utilizar en cada rol.

Para el servidor web, Apache HTTP Server es el estándar de referencia histórico en el mundo del software de código abierto, con el mayor despliegue acumulado a nivel mundial (Netcraft, 2024). Su arquitectura modular y el módulo de procesamiento MPM `event`, que gestiona conexiones mediante un modelo de hilos y eventos asíncronos, lo hacen representativo de lo que ACME encontraría en un entorno de producción real. Frente a alternativas como Nginx, Apache tiene mayor documentación académica y su comportamiento bajo carga concurrente está más ampliamente estudiado, lo que facilitará la interpretación de los resultados del benchmark.

Para el servidor de base de datos, PostgreSQL se seleccionó por dos razones complementarias. La primera es su robustez en producción: cumple los estándares ACID y es reconocido como uno de los motores

relacionales más sólidos del ecosistema open-source. La segunda razón es metodológica: PostgreSQL expone, en comparación con MySQL o MariaDB, una mayor variedad de operaciones que permiten ejercitar los subsistemas de CPU y almacenamiento del hipervisor de forma diferenciada. Operaciones como ordenamientos sobre grandes volúmenes, agregaciones estadísticas con percentiles, joins analíticos y funciones matemáticas de punto flotante intensivas en FPU cubren dimensiones del rendimiento que una prueba simple de INSERT no alcanzaría. Esta variedad es lo que hará posible construir un benchmark con mayor profundidad comparativa entre Proxmox y VirtualBox.

2.3 La definición del modo de red como decisión del equipo

El proyecto exige que las tres VMs, gestionadas por dos hipervisores distintos en dos equipos físicos diferentes, se comuniquen entre sí. La configuración de red no estaba prescrita: el equipo debía analizarla y resolverla. Esta fue la decisión técnica más analítica del proceso de diseño.

El marco teórico describe cuatro modos de red disponibles. El equipo descartó tres de ellos tras contrastarlos con los requisitos concretos del proyecto. NAT fue el primero en quedar fuera: aunque permite que una VM acceda a Internet, no la expone en la red local, por lo que los servicios de Apache y PostgreSQL no serían alcanzables desde otras máquinas sin configurar port forwarding manual por cada puerto, lo que resulta impráctico cuando hay múltiples servicios y múltiples clientes. La red interna (Internal Network) fue descartada porque opera únicamente dentro del mismo hipervisor, lo que impide por diseño la comunicación entre la VM de Proxmox y la de VirtualBox, que corren en equipos físicos distintos. Host-Only fue descartado porque aísla la red del entorno exterior, impidiendo igualmente la comunicación entre los dos equipos.

El modo Adaptador Puente (Bridge) es la única opción que resuelve todos los requisitos al mismo tiempo. Bridge opera en capa 2 del modelo OSI y conecta la interfaz virtual de la VM directamente al switch físico de la red, haciendo que cada VM aparezca como un nodo independiente con su propia dirección MAC e IP en la subred `192.168.137.0/24`. Al estar en la misma subred física, las VMs de Proxmox y VirtualBox podrán comunicarse de forma directa y transparente sin importar el tipo de hipervisor que las gestione. En Proxmox, este bridge se implementará a través de la interfaz `vmbr0`, que se crea automáticamente durante la instalación. En VirtualBox, se configurará el adaptador en modo Adaptador Puente sobre la interfaz Ethernet del equipo anfitrión. La VM Fedora Workstation utilizará además un segundo adaptador en modo NAT exclusivamente para acceso a Internet, combinando ambos modos para satisfacer simultáneamente los dos requisitos de conectividad.

2.4 El diseño del benchmark como solución a una limitación del entorno

El proyecto requiere medir y comparar el rendimiento de ambos hipervisores bajo condiciones de carga equivalentes. El principio fundamental del benchmarking comparativo es la equivalencia de condiciones: las VMs evaluadas deben tener las mismas configuraciones de vCPU, RAM, sistema operativo y versiones de software para que la comparación sea válida. Sin embargo, el equipo identificó una limitación estructural que no es posible eliminar: los dos equipos físicos disponibles tienen hardware profundamente diferente, con una brecha de nueve generaciones entre sus procesadores (Intel i5-3470 de 2013 en el equipo Proxmox vs. Intel i5-12450H de 2022 en el equipo VirtualBox) y una diferencia de almacenamiento tan significativa como la que existe entre un HDD mecánico y un SSD NVMe.

La respuesta del equipo a esta limitación es controlar todas las variables que sí están bajo su control — mismo sistema operativo Ubuntu Server 24.04.4 LTS en ambas VMs, mismos 2 cores de vCPU, mismos 6 GB

de RAM y mismas versiones de Apache y PostgreSQL— y documentar explícitamente la diferencia de hardware para que los resultados puedan interpretarse correctamente. En lugar de ignorar esta variable o intentar compensarla artificialmente, el equipo la incorporará como parte del análisis, separando el efecto del hardware del efecto del tipo de hipervisor al momento de interpretar los resultados.

Para la ejecución del benchmark se desarrollarán dos scripts Python que correrán desde la MV Fedora Workstation, actuando esta como cliente externo y árbitro neutro de las mediciones. El primero, `benchmark_apache.py`, lanzará 2,000 peticiones HTTP hacia cada servidor utilizando 50 hilos concurrentes mediante `ThreadPoolExecutor`, midiendo el tiempo real de transferencia con `time.perf_counter()` para obtener RPS, latencia promedio, mínima, máxima y desviación estándar, además de registrar todos los códigos de estado recibidos. El segundo, `benchmark_postgresql.py`, se conectará directamente a cada instancia de PostgreSQL mediante `psycopg2` y ejecutará dos categorías de pruebas: pruebas de I/O que incluyen INSERT masivo de 300,000 filas sobre una tabla `UNLOGGED` con valores UUID y texto generados aleatoriamente, lectura secuencial con agregaciones, creación de índice y cinco lecturas por rangos indexados; y pruebas de CPU que incluyen ordenamiento de 500,000 valores aleatorios, agregaciones estadísticas con percentiles 50, 95 y 99, join analítico entre dos conjuntos generados en memoria, y funciones matemáticas FPU-intensivas combinando raíz cuadrada, logaritmo natural y potencias. Cada prueba de CPU se repetirá tres veces para promediar los tiempos y reducir la varianza por carga puntual del sistema. Usar Fedora como cliente externo garantizará que el benchmark mida el rendimiento real desde la perspectiva de un usuario final, sin que los recursos del hipervisor evaluado interfieran en la medición.

---

3. Respuesta a las preguntas iniciales

3.1. Nivel Conceptual

3.1.1. Diferencias fundamentales entre hipervisores de Tipo 1 y Tipo 2

Un hipervisor de Tipo 1 (bare-metal) como Proxmox VE se instala directamente sobre el hardware del servidor, sin sistema operativo intermedio lo cual le permite acceder directamente a los recursos físicos (CPU, RAM, disco), reduciendo la sobrecarga que conlleva la virtualización. Por su parte, un hipervisor de Tipo 2 como Oracle VirtualBox se ejecuta sobre un sistema operativo anfitrión (en el proyecto se usó Pop!_OS 24.04 LTS), lo que introduce una capa adicional de abstracción entre la VM y el hardware.

En términos de rendimiento, bajo hardware idéntico el Tipo 1 debería tener una ventaja marginal por su acceso directo al hardware. Sin embargo, en el proyecto se evidenció que el hardware del equipo anfitrión fue el factor dominante puesto que el equipo VirtualBox superó en rendimiento al equipo Proxmox en todas las pruebas, no por diferencias en el tipo de hipervisor, sino por la brecha generacional del hardware en el que se ejecutaba cada tipo de hipervisor.

Por lo que se logró comprender que la elección del hipervisor no es una decisión única para toda la infraestructura, sino que puede variar según el rol de cada servidor dentro del proyecto.

3.1.2. Impacto en seguridad y aislamiento

Se logró evidencia que el hipervisor Tipo 1 es más seguro en términos de aislamiento, esto debido a que en VirtualBox, el SO anfitrión (Pop!_OS) actúa como intermediario entre las VMs y el hardware, por lo que si ese

SO es comprometido, todas las VMs quedan expuestas, mientrar que en Proxmox no existe ese intermediario, el hipervisor corre directamente sobre el hardware, por lo que un atacante que logre comprometer una VM se encontrará de frente con el hipervisor bare-metal, sin poder escalar al SO anfitrión debido a que no existe. A esto se suma que Proxmox permite configurar reglas de firewall individuales por VM desde su interfaz web, como se hizo con el bridge vmbr0 en el proyecto, añadiendo una capa de control de red adicional.

Lo anterior nos guía al hecho de que la mejor decisión es asignar los servidores de producción al hipervisor Tipo 1, donde el aislamiento es más robusto y la superficie de ataque es menor.

3.1.3. Implicaciones en la gestión y administración

Proxmox ofrece una interfaz web centralizada (`https://IP:8006`) la cual permite gestionar todas las VMs, monitorear recursos en tiempo real, crear snapshots, clonar VMs y gestionar almacenamiento desde un único panel, por su parte VirtualBox, al ser Tipo 2, depende del escritorio del SO anfitrión y su gestión es más manual e interactiva. En producción empresarial, Proxmox escala mejor al no requerir un SO base subyacente y al centralizar la administración de toda la infraestructura virtualizada.

Esta diferencia en la capacidad de gestión fue determinante para entender por qué Proxmox resulta más adecuado en escenarios donde se administran múltiples servidores simultáneamente desde un único punto de control.

3.1.4. Proxmox VE como plataforma Tipo 1: componentes principales

Proxmox VE está basado en Debian GNU/Linux e integra dos tecnologías de virtualización las cuales son KVM (Kernel-based Virtual Machine) para virtualización completa de hardware, y LXC (Linux Containers) para contenedores ligeros. En el proyecto, el componente de red fundamental fue el bridge vmbr0, a través del cual la VM "ServidorWEB" obtuvo conectividad con la red `192.168.137.0/24`, adicionalmente, la interfaz web de administración, accesible por el puerto 8006, permitió crear, configurar y monitorear la VM sin necesidad de interactuar directamente con la consola del servidor.

Comprender estos componentes fue fundamental para configurar correctamente la conectividad de red entre las VMs del proyecto, ya que el bridge vmbr0 operó como el punto de integración entre la VM y la red física del laboratorio.

3.2. Nivel Técnico

3.2.1. Instalación y configuración inicial de Proxmox y VirtualBox

Para Proxmox se preparó una memoria USB booteable con la ISO de Proxmox VE 9.1, se configuró el BIOS del equipo HP para arrancar desde USB y se realizó la instalación gráfica definiendo hostname (`proxmox.local`), dirección IP, gateway y DNS. Al finalizar, el bridge `vmbr0` con IP estática quedó configurado en `/etc/network/interfaces`. Para VirtualBox, al tratarse de un hipervisor Tipo 2, simplemente se instaló como aplicación sobre Pop!_OS 24.04 LTS y las VMs se crearon desde su interfaz gráfica sin requerir arranque especial del equipo.

3.2.2. Modos de red: NAT, Bridge, Host-Only e Internal

En la práctica del proyecto se utilizó principalmente el modo Adaptador Puente (Bridge), tanto en Proxmox como en VirtualBox. Este modo asigna a la VM una IP en la misma subred que el equipo anfitrión, permitiendo comunicación directa con otros equipos de la red física. La MV Fedora Workstation utilizó dos adaptadores simultáneamente, los cuales fueron adaptador Puente para acceder a la red interna `192.168.137.x` y NAT para acceder a Internet. El modo Bridge fue la clave para que las tres VMs gestionadas por dos hipervisores distintos se comunicaran fluidamente en la red `192.168.137.0/24`.

Modo de red	Acceso LAN	Acceso Internet	Acceso desde red externa	Uso en el proyecto
NAT	No	Sí	No	Fedora (adaptador 2, Internet)
Bridge	Sí	Sí (vía gateway)	Sí	Todas las VMs (comunicación interna)
Host-Only	Solo con el host	No	No	No utilizado
Internal	Solo entre VMs	No	No	No utilizado

3.2.3. Instalación y configuración de Apache

Se instaló el servidor web Apache en la MV de Proxmox mediante:

```
sudo apt install apache2 -y
sudo systemctl status apache2
```

Apache quedó escuchando en el puerto 80. Se verificó el acceso remoto desde Fedora Workstation navegando a `http://192.168.137.7`, donde se mostró correctamente la página por defecto de Apache2 Ubuntu, confirmando la conectividad entre hipervisores distintos.

3.2.4. Interconexión de VMs gestionadas por distintos hipervisores

La clave para lograr la comunicación entre la VM de Proxmox y la VM de VirtualBox fue que ambas se conectaron a la misma red física mediante el modo Adaptador Puente. La VM de Proxmox usó la interfaz `vmbr0` en modo bridge sobre la interfaz física del servidor, y la VM de VirtualBox también usó su interfaz en modo bridge sobre la tarjeta Ethernet del equipo anfitrión. Al estar en la misma subred `192.168.137.0/24`, la comunicación bidireccional funcionó de forma transparente, independientemente del tipo de hipervisor, por medio de esto se demuestra el hecho de que la interoperabilidad entre hipervisores distintos no requiere configuraciones especiales cuando se emplea Bridge networking sobre una red física común.

3.2.5. Instalación y configuración de PostgreSQL

Se instaló PostgreSQL en ambas VMs con el mismo procedimiento:

```
sudo apt install postgresql postgresql-contrib -y
sudo systemctl status postgresql
```

Para habilitar el acceso remoto se modificaron dos archivos de configuración:

- `/etc/postgresql/16/main/postgresql.conf` donde se modificó la línea para queda así: `listen_addresses = '*'`
- `/etc/postgresql/16/main/pg_hba.conf` se agregó la regla para la subred `192.168.137.0/24` con autenticación `scram-sha-256`

Finalmente se reinició el servicio con `sudo systemctl restart postgresql`. Se crearon bases de datos y usuarios específicos los cuales fueron `testing/nico` en la VM de Proxmox y `electiva/luis` en la VM de VirtualBox.

3.2.6. Direccionamiento IP y configuración de interfaces de red

La red implementada fue `192.168.137.0/24`. Para la VM de VirtualBox, Ubuntu Server utiliza DHCP por defecto, por lo que se debió configurar IP estática mediante Netplan editando `/etc/netplan/00-installer-config.yaml` el cual quedó de la siguiente forma:

```yaml
network:
  version: 2
  ethernets:
    enp0s3:
      addresses:
        - 192.168.137.6/24
      routes:
        - to: default
          via: 192.168.137.1
      nameservers:
        addresses:
          - 8.8.8.8
          - 8.8.4.4
```

Los cambios se aplicaron con `sudo netplan apply` y se verificaron con `ip a`. En Proxmox, la IP estática de la VM quedó configurada directamente durante la instalación de Ubuntu Server mediante el instalador Subiquity.

3.2.7. Pruebas de conectividad entre VMs

Se verificó conectividad a nivel de aplicación a través de dos formas, primero verificando el acceso HTTP a Apache desde Fedora Workstation navegando a `http://192.168.137.7`, y conexión a PostgreSQL con DBeaver desde Fedora hacia ambos servidores (`192.168.137.6:5432` y `192.168.137.7:5432`).

3.3. Nivel Comparativo

3.3.1. Metodología aplicada para los benchmarks

Se desarrollaron dos scripts Python ejecutados desde la MV Fedora Workstation como cliente externo, midiendo el rendimiento bajo condiciones equivalentes: misma versión de SO (Ubuntu Server 24.04.4 LTS), misma cantidad de CPU virtual (2 cores), misma RAM (6 GB) y los mismos servicios instalados (Apache y PostgreSQL).

Los scripts fueron:

- `benchmark_apache.py` el cual genera 2,000 peticiones HTTP con 50 hilos concurrentes hacia cada servidor.
- `benchmark_postgresql.py` en el que se realizan pruebas intensivas de I/O (INSERT masivo, lecturas, índices) y CPU (sort, agregaciones, joins, operaciones matemáticas) con 300,000 y 500,000 filas respectivamente.

3.3.2. Resultados del benchmark de Apache

Métrica	Proxmox (Tipo 1)	VirtualBox (Tipo 2)	Diferencia
RPS (Req/s)	1,100.88	1,150.33	VirtualBox +4.5%
Latencia promedio (s)	0.0406	0.0388	VirtualBox 4.4% menor
Latencia mínima (s)	0.0082	0.0095	Proxmox 13.7% menor
Latencia máxima (s)	0.0813	0.0883	Proxmox 7.9% menor
Errores	0	0	Ambos: 100% confiabilidad

Las diferencias en Apache fueron mínimas (~4.5%), lo que indica que para cargas web ligeras ambos hipervisores son equivalentes.

3.3.3. Resultados del benchmark de PostgreSQL

Métrica	Proxmox (Tipo 1)	VirtualBox (Tipo 2)	Diferencia de tiempo (s)
IO — INSERT 300K filas (s)	5.6155	1.4767	Proxmox +280.3%
IO — Seq scan (s)	0.0407	0.0370	Proxmox +10.0%
IO — Crear índice (s)	0.1592	0.1176	Proxmox +35.4%
IO — Lecturas indexadas avg (s)	0.0214	0.0198	Proxmox +8.1%
CPU — Sort avg (s)	0.2279	0.1747	Proxmox +30.5%
CPU — Agregaciones avg (s)	0.3017	0.2234	Proxmox +35.0%
CPU — Join avg (s)	0.2965	0.1954	Proxmox +51.7%
CPU — Math avg (s)	6.4578	3.9814	Proxmox +62.2%

3.3.4. Interpretación de los resultados

Los resultados no invalidan la superioridad teórica del Tipo 1, puesto que la diferencia se explica por el hardware en el que se ejecuta cada hipervisor:

Aspecto	Equipo Proxmox (Tipo 1)	Equipo VirtualBox (Tipo 2)
Procesador	Intel Core i5-3470 (3.ª gen, 2013)	Intel Core i5-12450H (12.ª gen, 2022)
Frecuencia	3.20 GHz	4.40 GHz
Almacenamiento	HDD mecánico	SSD NVMe

La diferencia más notable fue en I/O de escritura masiva (+280.3%), explicada principalmente por la brecha entre HDD y SSD NVMe. Las diferencias en CPU (entre +30% y +62%) son coherentes con la brecha generacional entre ambos procesadores debido a que el equipo de virtualbox es 9 generaciones más reciente. Bajo hardware idéntico, se esperaría que Proxmox (Tipo 1) sea igual o ligeramente más eficiente, dado que elimina la capa del SO anfitrión y gestiona los recursos con menor sobrecarga.

Esta distinción tiene implicaciones directas sobre cómo debe planificarse una migración de infraestructura, puesto que los beneficios de la virtualización en términos de rendimiento solo se materializan plenamente cuando el hardware anfitrión es adecuado para la carga de trabajo. Migrar a un hipervisor Tipo 1 sobre hardware envejecido puede resultar en rendimiento inferior al del servidor físico original, mientras que la misma migración sobre hardware moderno con almacenamiento SSD produciría resultados opuestos. Esta distinción fue clave para que el equipo pudiera interpretar los resultados del benchmark con rigor, evitando atribuir al tipo de hipervisor diferencias de rendimiento que en realidad se originan en el hardware subyacente.

3.4. Nivel Estratégico

3.4.1. Criterios de selección del hipervisor según tipo de servidor

La selección del hipervisor adecuado depende del rol que cumple cada servidor dentro de la infraestructura, considerando factores técnicos, operativos y económicos. A partir de lo observado durante el proyecto, se plantean las siguientes recomendaciones:

Servidor	Hipervisor recomendado	Justificación
Servidor Web (producción)	Proxmox (Tipo 1)	Mayor estabilidad, gestión centralizada, sin dependencia de SO anfitrión
Servidor BD (producción)	Proxmox (Tipo 1)	En hardware moderno con SSD, el acceso directo al hardware reduce latencia de I/O
Servidor de Desarrollo	VirtualBox (Tipo 2)	Facilidad de uso, snapshots rápidos, portabilidad entre equipos de desarrolladores

3.4.2. Costos de mantenimiento y soporte

La optimización de costos es uno de los argumentos centrales para justificar una migración hacia entornos virtualizados. Proxmox VE es open-source con suscripción empresarial opcional; sin suscripción funciona

completamente pero recibe actualizaciones desde el repositorio pve-no-subscription. Al no requerir licenciamiento por servidor virtualizado, permite consolidar múltiples servidores físicos en un único equipo sin incrementar los costos de software, reduciendo además los gastos asociados a energía, espacio físico y mantenimiento de hardware, sin embargo, el costo principal asociado es el hardware dedicado bare-metal y la curva de aprendizaje inicial para la administración del sistema. VirtualBox, por su parte, es gratuito para uso personal y evaluación, pero en entornos productivos deben considerarse varios costos adicionales: el SO anfitrión (licencias Windows si aplica), el Extension Pack el cual bajo su licencia PUEL requiere licencia comercial de Oracle para uso empresarial.

3.4.3. Recomendación estratégica para ACME

Para los servidores críticos de ACME (web y base de datos) se recomienda Proxmox VE sobre hardware moderno con almacenamiento SSD, garantizando la eliminación de la capa del SO anfitrión, gestión centralizada de la infraestructura y mejor relación rendimiento/costo a largo plazo. VirtualBox queda apropiado para el entorno de desarrollo, donde la portabilidad, la facilidad de administración por parte de los desarrolladores y la flexibilidad de los snapshots son más valiosas que el rendimiento máximo.

3.5. Nivel Reflexivo

3.5.1. Retos técnicos y de compatibilidad encontrados

Reto encontrado	Causa identificada	Solución aplicada
Benchmarks con resultados difíciles de interpretar directamente	Hardware anfitrión significativamente diferente entre los dos equipos	Análisis separado del efecto del hardware vs. efecto del tipo de hipervisor
Nombre de interfaz de red variable (`enp0s3`)	El nombre de la interfaz depende del entorno (VirtualBox vs bare-metal)	Verificación con `ip a` antes de editar Netplan
PostgreSQL no acepta conexiones remotas por defecto	Configuración restrictiva de `listen_addresses` y `pg_hba.conf`	Modificación de `listen_addresses = '*'` y adición de regla de subred en `pg_hba.conf`

3.5.2. Lecciones aprendidas

- La interoperabilidad entre hipervisores de distinto tipo es posible sin configuraciones complejas, siempre que ambas VMs compartan la misma red física a través de Bridge networking. El tipo de hipervisor no es una barrera para la comunicación entre máquinas virtuales.
- Para benchmarks comparativos válidos entre hipervisores, el hardware anfitrión debe ser controlado como variable. De lo contrario, los resultados reflejan diferencias de hardware más que del software de virtualización, abriendo la posibilidad de llegar a conclusiones incorrectas sobre qué plataforma conviene adoptar.
- Proxmox VE es una plataforma robusta y accesible para entornos académicos y productivos de escala media, con una curva de aprendizaje manejable gracias a su interfaz web y documentación oficial. La

instalación bare-metal requiere dedicar el equipo completo al hipervisor, lo cual debe planificarse con anticipación.

- La virtualización mixta (Tipo 1 + Tipo 2) es viable en producción real, representando arquitecturas híbridas donde los servidores críticos se despliegan en bare-metal y los entornos de desarrollo y pruebas en hipervisores Tipo 2 sobre equipos de usuario.
- La diferencia entre HDD y SSD tiene mayor impacto en el rendimiento de bases de datos que cualquier otra variable del sistema. En operaciones de escritura masiva, la brecha puede superar el 280%, lo que refuerza que la selección del hipervisor y la modernización del hardware son decisiones que deben tomarse de forma conjunta, no independiente.

3.6. Respuesta a la pregunta inicial

¿Cómo puede la empresa ACME migrar su infraestructura basada en servidores físicos hacia un entorno virtualizado para optimizar costos y disponibilidad, considerando la selección del hipervisor, las herramientas de gestión de VMs y la conectividad de red interna?

La evidencia obtenida a lo largo del proyecto permite entender que ACME no debe elegir entre un tipo u otro de forma absoluta, sino asignar cada hipervisor según el rol del servidor. Proxmox VE es la plataforma adecuada para los servidores de producción (web y base de datos), esto debido a que al instalarse directamente sobre el hardware elimina la capa del SO anfitrión, reduce la superficie de ataque, centraliza la administración en una interfaz web y no requiere licenciamiento comercial, por otra parte, VirtualBox es la plataforma adecuada para el servidor de desarrollo, a causa de su facilidad de instalación, portabilidad entre equipos y herramientas como los snapshots rápidos ajustándose a las necesidades del equipo de desarrollo sin justificar el costo y la complejidad de un hipervisor bare-metal. El factor más importante que esta investigación añade a esa decisión es que el hardware anfitrión pesa más que el tipo de hipervisor en el rendimiento final, puesto que migrar a Proxmox sobre un equipo con HDD y procesador de tercera generación no garantiza mejora frente al servidor físico original, por lo que la modernización del hardware y la virtualización deben planificarse de forma conjunta.

Respecto a las herramientas de gestión de máquinas virtuales, Proxmox ofrece de forma nativa las capacidades que ACME requiere para mejorar la disponibilidad frente a su infraestructura física actual, tales como snapshots para capturar el estado de una VM antes de realizar cambios de riesgo, clonación para replicar entornos en minutos, monitoreo de CPU y RAM en tiempo real desde un panel web centralizado, y la posibilidad de gestionar múltiples servidores virtuales desde un único punto de administración. Estas funcionalidades no están disponibles de forma integrada en servidores físicos tradicionales representando una de las principales ventajas a nivel operativo de la migración. Por su parte, VirtualBox también dispone de herramientas como snapshots, clonación de máquinas virtuales y monitoreo básico de recursos desde su interfaz gráfica, sin embargo, estas están orientadas a entornos individuales de desarrollo y pruebas, a diferencia de Proxmox, no ofrece de manera nativa una gestión centralizada multiusuario ni administración remota con enfoque empresarial, ya que depende del sistema operativo anfitrión y está diseñado principalmente para su uso en estaciones de trabajo, por lo tanto, ambas herramientas resultan complementarias dentro de la arquitectura propuesta, donde Proxmox aporta capacidades de gestión a nivel productivo y VirtualBox responde a las necesidades de flexibilidad y experimentación del entorno de desarrollo.

En cuanto a la conectividad de red interna, la interconexión entre los servidores virtuales de ACME no depende del tipo de hipervisor que los gestione, ya que el proyecto demostró que una VM gestionada por

Proxmox y otra por VirtualBox pueden comunicarse de forma transparente cuando ambas usan el modo Adaptador Puente sobre la misma red física. Para ACME, esto significa que la migración puede hacerse de forma gradual donde los servidores ya migrados a VMs y los servidores físicos que aún no han sido migrados pueden coexistir en la misma red sin interrupciones de servicio, lo que reduce el riesgo operativo de la transición.

Por lo que la migración de ACME es técnicamente viable, económicamente justificable con Proxmox como plataforma principal, y operativamente segura si se planifica en fases que combinen la modernización del hardware con la virtualización progresiva de los servidores, comenzando por los de producción en Proxmox y manteniendo el entorno de desarrollo en VirtualBox.

4. Conclusiones

La diferencia entre un hipervisor Tipo 1 y uno Tipo 2 va más allá de una capa de software adicional: implica modelos de seguridad y administración distintos. Proxmox elimina el SO anfitrión como vector de ataque y centraliza la gestión en una interfaz web, mientras que VirtualBox gana en portabilidad y accesibilidad a costa de heredar las vulnerabilidades del sistema sobre el que corre. Ninguna arquitectura es universalmente superior; la elección adecuada depende del rol del servidor y los requisitos del entorno.

La conectividad entre VMs gestionadas por hipervisores distintos no es un problema de compatibilidad entre plataformas sino un problema de capa de red. El modo Bridge lo resuelve porque opera en capa 2 del modelo OSI, antes de que el tipo de hipervisor tenga relevancia, lo que hace que esta solución sea aplicable a cualquier escenario de infraestructura mixta.

Proxmox VE demuestra que una plataforma de virtualización open-source puede ser técnicamente comparable a soluciones comerciales. La combinación KVM+QEMU+VirtIO reduce el overhead de CPU e I/O a niveles que hacen viable su uso en producción real sin licenciamiento comercial, lo que lo convierte en una alternativa legítima frente a VMware ESXi o Citrix Hypervisor para organizaciones con restricciones de presupuesto como ACME.

En cualquier benchmark comparativo, las variables que no se pueden controlar deben documentarse explícitamente y formar parte del análisis. La diferencia de hardware entre los dos equipos disponibles no invalida la comparación, pero sí condiciona la interpretación: sin ese reconocimiento, los resultados llevarían a conclusiones incorrectas sobre los hipervisores en lugar de sobre el hardware que los soporta.

Referencias

Adams, K., & Agesen, O. (2006). A comparison of software and hardware techniques for x86 virtualization. *ACM SIGPLAN Notices, 41*(11), 2–13. https://doi.org/10.1145/1168857.1168860

Bartholomew, D. (2006). QEMU: A multihost, multitarget emulator. *Linux Journal, 2006*(145), 2.

Blackburn, S. M., Diwan, A., Hauswirth, M., Sweeney, P. F., Amaral, J. N., Brecht, T., ... & Wiedermann, B. (2016). The truth, the whole truth, and nothing but the truth: A pragmatic guide to assessing empirical evaluations. *ACM Transactions on Programming Languages and Systems, 38*(4), 15. https://doi.org/10.1145/2983574

Creasy, R. J. (1981). The origin of the VM/370 time-sharing system. *IBM Journal of Research and Development, 25*(5), 483–490. https://doi.org/10.1147/rd.255.0483

Goldberg, R. P. (1972). Architectural principles for virtual computer systems [Doctoral dissertation, Harvard University].

Gray, J. (Ed.). (1993). *The benchmark handbook: For database and transaction processing systems* (2nd ed.). Morgan Kaufmann.

Kivity, A., Kamay, Y., Laor, D., Lublin, U., & Liguori, A. (2007). KVM: The Linux virtual machine monitor. *Proceedings of the Linux Symposium, 1*, 225–230.

Laureano, M., & Maziero, C. (2008). Virtualização: Conceitos e aplicações em segurança. *VIII Simpósio Brasileiro em Segurança da Informação e de Sistemas Computacionais (SBSeg)*, 1–50.

Netcraft. (2024). *Web server survey*. https://www.netcraft.com/blog/

Oracle Corporation. (2024). *Oracle VM VirtualBox user manual* (version 7.x). https://www.virtualbox.org/manual/

Popek, G. J., & Goldberg, R. P. (1974). Formal requirements for virtualizable third generation architectures. *Communications of the ACM, 17*(7), 412–421. https://doi.org/10.1145/361011.361073

Portnoy, M. (2012). *Virtualization essentials*. Sybex/Wiley.

PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/

Proxmox Server Solutions GmbH. (2024). *Proxmox VE administration guide* (version 8.x). https://pve.proxmox.com/pve-docs/

Russell, R. (2008). virtio: Towards a de-facto standard for virtual I/O devices. *ACM SIGOPS Operating Systems Review, 42*(5), 95–103. https://doi.org/10.1145/1400097.1400108

Smith, J. E., & Nair, R. (2005). The architecture of virtual machines. *IEEE Computer, 38*(5), 32–38. https://doi.org/10.1109/MC.2005.173

Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4th ed.). Pearson Education.

Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer networks* (5th ed.). Pearson Education.